10mg Health

10MG HEALTH · TECHNICAL ASSESSMENT

Multi-Tenant Lending Core Architecture & Database Design

How one shared, white-label backend lets many lenders offer fast, data-driven credit to Africa's pharmacies, clinics and hospitals, with strict tenant isolation and auditable money movement.

16
domain tables (MySQL 8.4)

48
documented API endpoints (39 URLs)

260
automated tests · 1,151 assertions

151/151
adversarial probe checks, incl. real concurrency

Laravel 13 · PHP 8.3 · Sanctum · MySQL · Redis

Version 1.3 · November 2026

Contents

1 Context & goals

2 System architecture
Components · request pipeline · deployment

3 Multi-tenancy design

4 Authentication & access control

5 Lending workflow
State machine · credit scoring · pricing maths · BVN / NIN KYC

6 Financial integrity
10MG platform fee · race conditions

7 API conventions & error model

8 Database design
Principles · ERD · lifecycle of a loan in data

9 Data dictionary
Every table, column, index and foreign key

10 Verification

11 Trade-offs & roadmap

1 Context & goals

10MG Health provides credit to healthcare providers whose cash needs move faster than traditional bank lending: a pharmacy restocking antimalarials before rainy season, a clinic replacing a broken ultrasound scanner, or a hospital bridging payroll so emergency care keeps running.

This backend is a simplified **10MG V2 lending core**. It's a single deployment that many financial institutions (banks, microfinance banks, credit unions) use as **tenants**. Each tenant white-labels the platform under its own brand, sets its own loan products and pricing, runs its own staff, and lends to its own borrowers. No tenant can see another's data.

Requirement	How the design meets it
Multi-tenant architecture	Shared schema with row-level isolation that fails closed. Institution onboarding and approval. Sanctum token auth. Permission-based RBAC with enforced maker/checker.
White-label configuration	Per-tenant branding and custom domain. Products with limits, interest method and rate, frequency, grace and late fees, minimum score and eligible provider types. Upfront or capitalised fees.
Lending workflow	Owner identity check (BVN via NIBSS, NIN via NIMC) → apply → instant business-profile scoring (mock) → approve/reject → disburse (mock rail) → waterfall repayments → automatic closure. Daily overdue and penalty job.
Financial integrity	10MG platform fee per lender (lender- or borrower-borne), snapshotted per loan and recorded in an append-only revenue ledger. Integer minor units and bcmath. Transactions and row locks. HTTP and domain idempotency. Append-only ledger and audit trail. Encrypted, masked BVN/NIN. Layered validation and a stable error envelope.
Code quality	Controller → Service → Repository layering, enforced by architecture tests. Contracts at integration boundaries. 260 automated tests. OpenAPI 3.1 docs. This document.

Borrowers are businesses

Pharmacies, patent medicine stores, clinics, hospitals, laboratories and diagnostic centres. Each is scored on its trading profile, not on personal salary.

Lenders are tenants

Every row of tenant data belongs to exactly one institution. Isolation is enforced centrally, never left to individual queries.

Money is never a float

Kobo integers, basis-point rates, exact rounding. Every naira can be traced through the ledger and audit trail.

2 System architecture

The system is a **modular monolith** on Laravel 13 with a strict **Controller → Service → Repository** layering. HTTP concerns (authentication, tenancy, authorisation, idempotency) are handled once in the middleware pipeline. Controllers hold no logic. **Services** own every business rule and transaction, and reach the database only through **repository interfaces**, whose Eloquent implementations own all queries and row locks. External dependencies (credit scoring, the payment rail, and the NIBSS and NIMC identity registers) sit behind contracts. The scorer and rail are mocks; the identity clients make real HTTP calls in `live` mode and answer locally in `sandbox` mode. Any of them can be swapped for a production provider without touching business code. An architecture test fails the build if a controller or service breaks these rules.

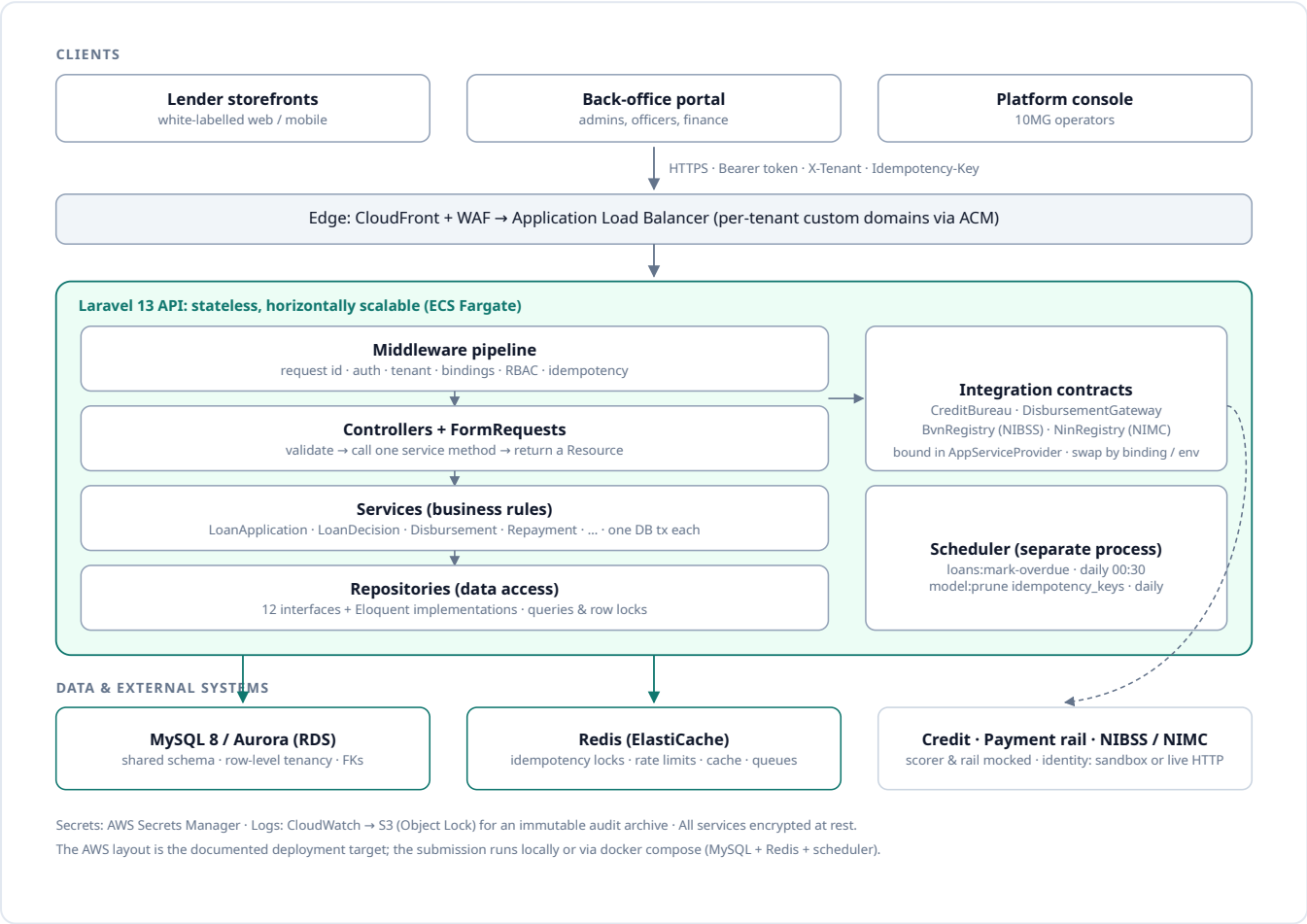


Figure 1: Component view and target AWS deployment.

Request pipeline

Every API request passes through the same ordered pipeline. Ordering is enforced with Laravel's middleware priority list. The tenant is resolved *before* route-model binding, so a foreign record never resolves and the request returns 404. The idempotency key is claimed only *after* authorisation passes.

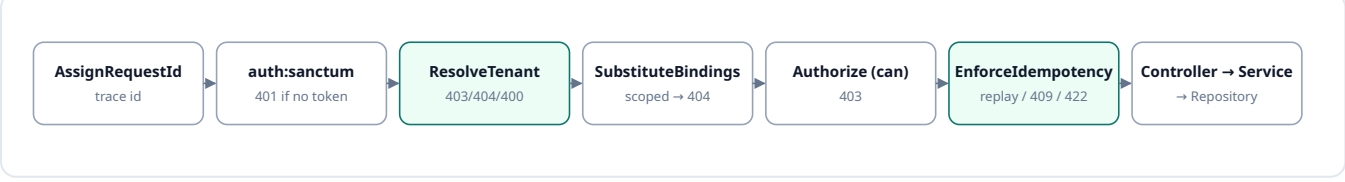


Figure 2: Middleware order (highlighted stages are custom: tenancy and idempotency).

Code organisation

Layer	Location	Responsibility
HTTP edge	`Http/Middleware, Http/Requests`	Request id, tenant resolution, idempotency; input validation (FormRequests).
Controllers	`Http/Controllers/Api/V1/*`	No logic: FormRequest in → one service call → API Resource out.
Services	`Services/*Service, Services/Loans/*`	Every business rule, transaction, invariant and audit write. Data access only through repository interfaces.
Repositories	`Repositories/Contracts, Repositories/Eloquent`	All queries, filters, pagination and row locks; bound in `RepositoryServiceProvider`, mockable in tests.
Domain helpers	`Services/Lending, Services/AuditLogger`	Pure pricing engine; audit writer.
Integration contracts	`Contracts/, Services/Credit, Services/Disbursement, Services/Identity`	`CreditBureau`, `DisbursementGateway`, `BvnRegistry` (NIBSS), `NinRegistry` (NIMC).
Tenancy	`Support/Tenancy`	`TenantContext`, `InstitutionScope`, `BelongsToInstitution`.
Policy & RBAC	`Enums/Role, Enums/Permission, Policies`	Role → permission matrix registered as Gates; `LoanPolicy` for borrower self-service.
Errors	`Exceptions/Api/*, ApiErrorRenderer`	One class per machine-readable error code; single JSON envelope.

3 Multi-tenancy design

Strategy: shared database, shared schema, row-level isolation on `institution_id`. This suits a white-label platform with many small and medium lenders. It needs a single migration path, pools connections well, and costs little per tenant to run.

Option	Isolation	Operational cost	Chosen
Database per tenant	Strongest	High: N migrations, N connection pools	
Schema per tenant	Strong	Medium; not portable to MySQL	
Shared schema + institution_id	Enforced centrally in the app, backed by FKs and composite unique keys	Low; scales to many tenants	✓

Defense in depth

1. **Tenant resolution.** For an authenticated user, the tenant always comes from `users.institution_id` on the token, so it can't be spoofed. Anonymous storefront calls use the `X-Tenant: <slug>` header. A header that conflicts with the token returns `403 TENANT_MISMATCH`. Platform operators have no tenant and can't call tenant routes (`400 TENANT_REQUIRED`).
2. **Global scope that fails closed.** Every tenant-owned model uses `BelongsToInstitution`, which adds `WHERE institution_id = :current` to every query, including route-model binding. With no tenant in context the scope matches *nothing*. Cross-tenant system jobs (the overdue command) must opt in explicitly with `TenantContext::runAs()`.
3. **Safe writes.** New tenant records are stamped with the current tenant automatically. Creating tenant data with no tenant in context throws.
4. **Existence is never revealed.** A cross-tenant ID returns `404`, not `403`, so one lender can't even confirm another's records exist.
5. **Per-tenant uniqueness.** Business keys are unique *within* a tenant: user email, borrower CAC number and phone, product code. The same pharmacy can bank with two lenders.
6. **Tenant lifecycle.** Pending or suspended lenders are refused (`403 TENANT_INACTIVE`). Suspending a lender revokes every token its users hold.

> **Why not rely on developers adding WHERE clauses?** One forgotten filter is a data breach. The scope is applied by the framework to every query, and its default when the tenant is unknown is deny. Tests check cross-tenant reads, writes, header spoofing and the fail-closed default.

4 Authentication & access control

Authentication uses **Laravel Sanctum** bearer tokens: hashed at rest, revocable, and expiring after 30 days. Roles map to fine-grained permissions in one enum. Each permission is registered as a Gate, and routes declare `->can('loans.review')`. Controllers never check roles directly.

Permission	platform admin	institution admin	loan officer	finance officer	customer (borrower)
institutions.manage (approve / suspend lenders)	✓				
audit.view	✓	✓			

Permission	platform admin	institution admin	loan officer	finance officer	customer (borrower)
branding.manage · kyc-settings.manage · products.manage · staff.manage		✓			
platform-fees.view (10MG fees owed by the lender)		✓		✓	
loans.view (all loans in tenant, borrower list)		✓	✓	✓	
loans.review (approve / reject)		✓	✓		
loans.disburse		✓		✓	
repayments.record (any channel, incl. cash)		✓		✓	
loans.apply · loans.view-own · repayments.make-own					✓

- **Maker/checker, enforced per loan.** Approval and disbursement are separate permissions. The user who approved a loan can't disburse it (`403 SEGREGATION_OF_DUTIES`), even an institution admin who holds both. It's configurable for single-operator lenders.
- **Borrower self-service.** `LoanPolicy` limits borrowers to their own loans. Borrowers can't self-report `cash` repayments. Their legal identity (CAC number, business type) can't be changed through self-service. Borrowers verify their owner's BVN and NIN themselves (`/me/kyc`); staff see the KYC status but never the numbers.
- **Account safety.** Deactivating a user removes their permissions and revokes their tokens. Admins can't deactivate themselves or change their own role. Login and onboarding are rate-limited.

5 Lending workflow

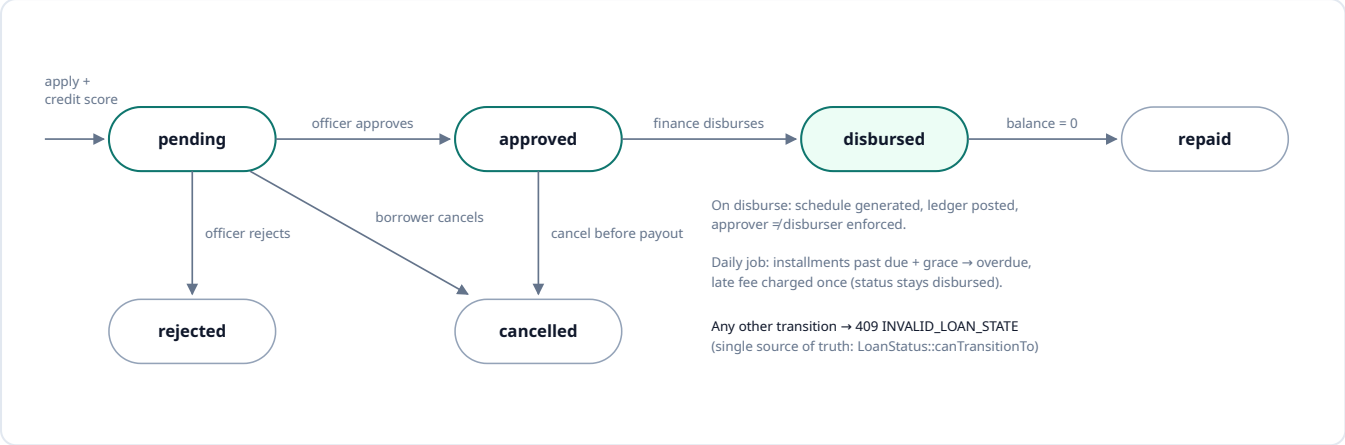


Figure 3: Loan state machine.

1. **Apply.** The borrower picks a product, amount, tenor and purpose (inventory restock, medical equipment, working capital, emergency financing, facility expansion). The request is checked against the product's limits and its eligible provider types. The product's terms (rate, method, frequency, fees, overdue rules) are **snapshotted onto the loan**, so later repricing never changes an existing loan.
2. **Instant credit assessment.** The business profile is scored synchronously and the explainable result is stored (below).
3. **Decision.** An officer approves or rejects. Approving a `decline` recommendation requires a written `override_reason`, which is kept on the loan and in the audit trail.
4. **Disburse.** A different authorised user releases the funds. Inside one transaction, the system calls the payment rail with a deterministic idempotency reference, generates the schedule from the payout date, and posts disbursement and fee ledger entries. A rejected transfer returns `502` and rolls everything back.
5. **Repay.** Payments are allocated oldest installment first, and within an installment **penalty → fees → interest → principal**. Partial payments are supported and overpayment is refused. The loan closes automatically at zero.
6. **Overdue.** A daily job marks installments past `due date + grace` as overdue and charges the snapshotted late fee once per installment, with a ledger entry.

Credit assessment: healthcare business scorecard

`MockHealthcareCreditScorer` stands in for 10MG's AI business-profile model behind the `CreditBureau` contract. It's deterministic and fully explainable: each factor's impact is stored, so a loan officer can see exactly why a recommendation was made.

Factor	Signal	Score impact
Operating licence	Licence from PCN, MDCN, MLSCN or State MoH, valid today	+40 valid · −200 expired (**forces decline**)
Debt-service coverage	Net monthly cash flow ÷ new monthly repayment (weekly normalised ×52/12)	≥2.0x +80 · ≥1.5x +30 · ≥1.2x −40 · <1.2x −150 (**forces decline**)
Years in operation	Current year − year established	<1 −60 · 1–2 −10 · 3–5 +30 · >5 +60
Provider type	Business type	Hospital +30 · pharmacy/clinic +20 · lab/diagnostic +10 · PPMV 0

Factor	Signal	Score impact
Revenue scale	Monthly revenue	≥₦5m +30 · ≥₦1m +10 · otherwise −20
Existing exposure	Open loans with this lender	−40 each (max −120)

Base score is 600, clamped to 300–850, graded A (≥750) to E. **Decline** if the licence is expired, coverage is below 1.20x, or the score is below the product minimum (default 560). **Approve** if the score is ≥680 and coverage is ≥1.50x. Otherwise **review**. The scorer also returns the largest principal that keeps coverage at 1.50x.

Pricing maths

- **Flat:** interest = P × annual rate × (n ÷ periods per year), spread evenly.
- **Reducing balance:** amortised EMI = $P{\cdot}r \div (1 - (1+r)^{-n})$, with r = annual rate ÷ periods per year (52 / 26 / 12). Each period's interest is charged on the outstanding principal.
- All arithmetic uses **bcmath** at 20-digit precision and is rounded half-up to kobo only when stored. Rounding residue goes into the **last installment**, so the schedule always sums exactly to the quoted totals. The engine was cross-checked against an independent Decimal implementation to the kobo.
- **Fees:** *upfront* fees are deducted from the payout (net disbursement = principal − upfront fees). *Capitalised* fees are added to the repayable amount and spread across installments.

Borrower KYC: BVN and NIN

The borrower is a business, but a BVN or NIN belongs to a **person**, so both are captured for the business's owner in `customer_directors`. Before the business can apply, the owner submits name, date of birth, phone, BVN and NIN to `POST /me/kyc` (with an `Idempotency-Key`, because each attempt is a billable lookup). Until it passes, `POST /loans` returns `422 KYC_REQUIRED`.

Layer	Class	Responsibility
Transport	`IdentityRegistryClient → NibssService, NimcService`	OAuth2 client-credentials token (cached 50 min), JSON request with timeout and retry on connection errors, provider envelope `{responseCode, reference, data.fieldMatches}`. Unreachable provider → `503 IDENTITY_PROVIDER_UNAVAILABLE`.
Matching rules	`BvnService, NinService`	Number not found → fail. First name, last name and date of birth must match. A phone mismatch is recorded but tolerated, because BVN phone numbers are often out of date.
Orchestration	`KycService`	Rejects a BVN/NIN already linked to another business of the same lender, calls both registers **outside** the transaction (no lock held on the network), then locks the borrower, re-checks, saves the owner and status, and audits, all in one transaction.

> **Sandbox and live modes.** With `NIBSS_MODE` / `NIMC_MODE` set to `sandbox` (the default), each service builds exactly the request it would send and answers it locally with a full match, so the whole flow runs without credentials. `live` sends it; the tests drive live mode through `Http::fake()` to pin the endpoints, headers and payloads. **Protecting the numbers (NDPA 2023):** encrypted at rest, an HMAC fingerprint for uniqueness, last four digits for display, masked (`*******8901`) in every response and audit entry, never serialised.

KYC SETTINGS: EACH LENDER'S CHOICE

Each lender sets its own policy through `GET` / `PATCH /kyc-settings` (institution admin, permission `kyc-settings.manage`, audited). Until it does, the platform defaults apply.

Switch (default)	Effect
`kyc_required (on)`	Applications are refused with `422 KYC_REQUIRED` and the `outstanding_checks` until every enabled check has passed. Off: verification is optional.
`bvn_check_enabled (on)`	Match the BVN with NIBSS. Off: no BVN is asked for and NIBSS is never called.
`nin_check_enabled (on)`	Match the NIN with NIMC. Off: likewise for the NIN and NIMC.
`phone_match_required (off)`	A phone mismatch fails the check instead of only being recorded.

Requiring KYC with both checks off is refused (`422`). With both off, `POST /me/kyc` answers `409 KYC_NOT_ENABLED` and the status is `not_required`. `customers.kyc_checks` records the checks a business has passed, and status is judged against the lender's **current** settings. A borrower verified under a BVN-only policy has `nin` outstanding once the lender switches it on, and resubmits; a check switched off keeps its earlier result. The storefront (`GET /tenant`) publishes the active checks so the white-label app asks only for what the lender needs, and platform operators see each lender's policy read-only in `GET /platform/institutions/{id}` (`kyc_settings`, alongside its `platform_fee` terms).

Next uses of the verified BVN: credit-bureau lookups (CRC, FirstCentral) in the scorer, NIP name enquiry on the payout account before disbursement, and Global Standing Instruction / direct-debit mandates for collections.

6 Financial integrity

Concern	Mechanism
Money representation	Unsigned `BIGINT` minor units (kobo). Rates and percentage fees in basis points. No floating point anywhere in the money path. Every API amount is capped at ₦10 trillion so 64-bit arithmetic can never overflow.
Atomicity	Each money-moving service method runs in one `DB::transaction()`. A failure rolls back the loan, schedule, ledger and audit rows together.
Concurrency	`SELECT … FOR UPDATE` on the loan row before any state or balance change. This prevents double disbursement and races between concurrent repayments. Applications lock the borrower row, so the open-loan limit can't be raced.
HTTP idempotency	`Idempotency-Key` is required on apply, approve, reject, cancel, disburse and repay. The key is scoped per user and fingerprints method + path + canonicalised body (SHA-256). The first response is stored, and a retry replays it with `Idempotent-Replayed: true`. The same key with a different payload returns `422`. A concurrent duplicate returns `409` (atomic Redis/cache lock). `5xx` responses aren't stored, so the client can safely retry.
Domain idempotency	`loan_transactions.external_reference` is unique per institution. A payment provider's reference can only ever be applied once (`409 DUPLICATE_PAYMENT_REFERENCE`), even with a fresh idempotency key.
Immutable ledger	`loan_transactions` is append-only: no `updated_at`, and the model refuses updates and deletes. Each entry stores `balance_before` and `balance_after` plus its allocation, so the loan balance can be rebuilt from the ledger alone.
Audit trail	`audit_logs` is append-only. Every privileged action records the actor, tenant, IP, user agent, `X-Request-Id` and old/new values. It's written inside the same transaction as the change, so a failed action leaves no audit entry, and a failed disbursement is audited separately after the rollback.
Invariants	Checked by tests and by direct SQL against MySQL: `outstanding_balance` = Σ installment outstanding = last ledger `balance_after` = `total_repayable − amount_paid`.

10MG platform fee

10MG earns a fee on every disbursed loan, separate from the lender's own product fees. 10MG never funds lenders: they lend their own money, and the platform fee is 10MG's revenue.

Aspect	Design
Who sets it	A platform operator, per lender: `PATCH /platform/institutions/{id}/platform-fee` with `rate_bps` (basis points of the principal, 0–1000; 0 waives it) and `bearer`. Lenders never set use the platform default (`PLATFORM_FEE_BPS=50` , `PLATFORM_FEE_BEARER=lender`). Audited as `institution.platform_fee_updated`.
Lender pays	A commission the lender owes 10MG. The borrower's quote, payout and repayments are unchanged, and borrowers never see it (staff see it on each loan as `platform_fee`).
Borrower pays	Priced as an upfront fee named *10MG platform fee*: listed in the quote, deducted from the payout (1% on ₦200,000 → ₦198,000 paid out) and posted to the loan ledger with the lender's other upfront fees.
Snapshot	`platform_fee_bps` , `platform_fee_bearer` and `platform_fee_amount` are copied onto the loan at application, so changing a lender's terms never reprices an existing loan.

Aspect	Design
Revenue ledger	At disbursement, in the same transaction, one append-only `platform_fees` row is written (unique per loan, so a loan is never charged twice). Operators report across lenders with `GET /platform/platform-fees` (filters `institution_id`, `from`, `to`; totals per bearer). Each lender's admin and finance officers see its own terms and fees with `GET /platform-fees`.

Race conditions

Each race was reproduced with real parallel requests against MySQL 8.4 and a 12-worker server, and each has a regression test.

Race	Guard	Observed under contention
Concurrent repayments overdrawing a loan	Loan row lock, then the overpayment check	12 sent, only 4 fit: 4 accepted, 8 × 422; balances exact
One payment retried in parallel	Atomic lock per (user, Idempotency-Key) + unique index	10 retries: one payment applied; the rest replayed or told to retry (409)
Two officers approve / two finance officers disburse	Loan row lock + state machine	1 success, 9 × 409; paid out once, one schedule
Parallel applications beyond the open-loan limit	Borrower row lock around count-then-insert	8 sent, limit 3: exactly 3 accepted
Two overdue jobs plus repayments	Scheduler overlap guard, per-loan lock, overdue installments skipped	Each installment penalised once
One borrower submitting KYC twice at once	Borrower row lock, then a re-check of `kyc_status`; unique (customer_id, role)	8 sent: 1 × 200, 7 × 409 KYC_ALREADY_VERIFIED
Two businesses submitting the same BVN	Unique (institution_id, bvn_hash); the violation is answered as 422 on `bvn`	Loser rolls back and stays pending
Duplicate sign-ups, slugs, product codes, staff emails, domains	Unique indexes are authoritative; the losing insert is answered with the same 422 a sequential duplicate gets (no 500, no SQL leak)	8 sent: 1 × 201, 7 × 422, exactly one row

Lock order is the same everywhere (loan → installments; borrower → new loan), so these paths can't deadlock. Accepted windows: product edits are last-write-wins (audited, and loans are protected by snapshots); a request that crashes mid-flight leaves its key "processing" (409 until expiry, never a double execution).

7 API conventions & error model

Base path	`/api/v1`. Versioned, with JSON resources wrapped in `data` and paginated lists.
Tenant	From the token for tenant users. `X-Tenant: <slug>` for the public storefront, borrower sign-up and login.
Money / rates	Integers in minor units; rates in basis points (`2400` = 24.00% p.a.).
Tracing	`X-Request-Id` is accepted or generated, echoed on every response, and stored on audit rows.
Documentation	OpenAPI 3.1, generated from the code (Scramble): interactive at `/docs/api`, exported to `docs/openapi.json`. Also a Postman collection.

Every failure uses one envelope: `{"message": "…", "error": {"code": "MACHINE_CODE", "details": {…}}}`.

Status	Code	Meaning
400	`TENANT_REQUIRED` · `IDEMPOTENCY_KEY_REQUIRED`	Missing tenant context / missing idempotency key
401	`UNAUTHENTICATED` · `INVALID_CREDENTIALS`	No/invalid token; wrong login
403	`FORBIDDEN` · `TENANT_MISMATCH` · `TENANT_INACTIVE` · `SEGREGATION_OF_DUTIES` · `ACCOUNT_DISABLED`	Not permitted
404	`NOT_FOUND` · `TENANT_NOT_FOUND`	Missing or belongs to another tenant
409	`INVALID_LOAN_STATE` · `DUPLICATE_PAYMENT_REFERENCE` · `IDEMPOTENCY_REQUEST_IN_PROGRESS` · `INVALID_INSTITUTION_STATE` · `KYC_ALREADY_VERIFIED` · `KYC_NOT_ENABLED`	Conflicts with current state
422	`VALIDATION_FAILED` · `LOAN_TERMS_VIOLATION` · `OVERPAYMENT` · `CREDIT_OVERRIDE_REQUIRED` · `OPEN_LOAN_LIMIT_REACHED` · `IDEMPOTENCY_KEY_REUSED` · `KYC_REQUIRED` · `KYC_VERIFICATION_FAILED`	Request understood but not acceptable
502	`DISBURSEMENT_FAILED`	Payment rail rejected the transfer (rolled back)
503	`IDENTITY_PROVIDER_UNAVAILABLE`	NIBSS or NIMC unreachable; nothing stored, safe to retry

8 Database design

Design principles

ULID primary keys

Sortable, generated without a round-trip, and impossible to enumerate. Raw IDs never reveal record counts or another lender's activity.

Tenant key everywhere

Every tenant-owned table has an indexed, FK-constrained `institution_id`, and business keys are unique per tenant.

Exact money

Unsigned BIGINT minor units; basis-point rates. Balances are maintained under row locks and can be rebuilt from the ledger.

Snapshots over references

Loans copy the product terms and fees they were priced with. Products can be repriced or archived (soft delete) safely.

Append-only history

Ledger and audit tables are never updated or deleted, and FKs from them use RESTRICT or SET NULL so history survives.

Indexes follow queries

Composite indexes back the hot paths, e.g. `(institution_id, status, created_at)` for loan queues and `(status, due_date)` for the overdue job.

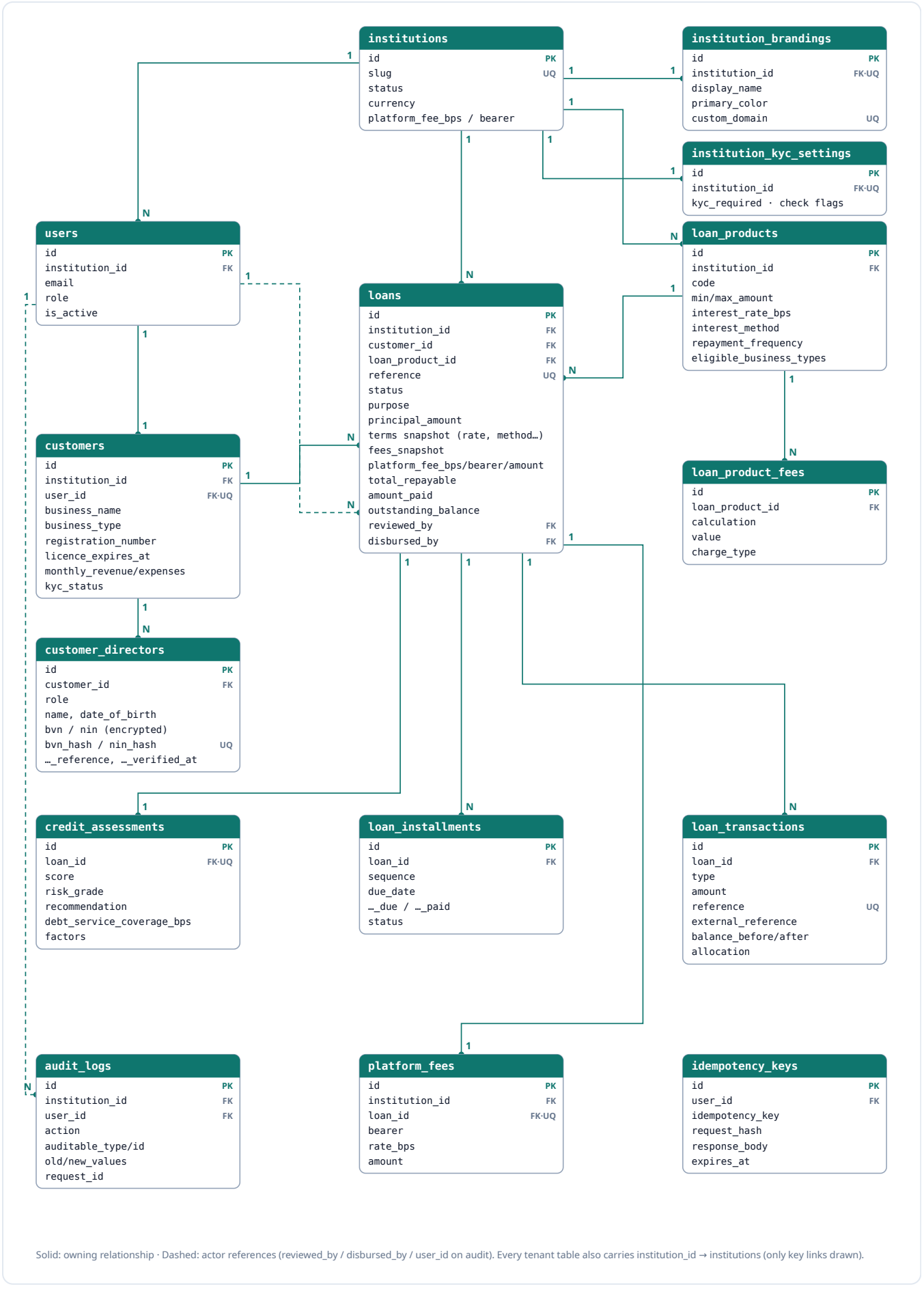


Figure 4: Entity-relationship diagram (key columns shown; full definitions in §9).

A loan's life in the data

Step	Rows written (one transaction each)
KYC	`customer_directors` (owner, encrypted BVN/NIN, references) + `customers` (kyc_status) + `audit_logs` (customer.kyc_verified / kyc_failed, masked numbers only)
Apply	`loans` (status pending, terms snapshot) + `credit_assessments` + `audit_logs` (loan.applied) + `idempotency_keys`
Approve / reject	`loans` (status, reviewed_by, decision_reason) + `audit_logs`
Disburse	`loans` (balances, dates, rail reference) + N × `loan_installments` + `platform_fees` (10MG fee) + `loan_transactions` (disbursement, upfront fee) + `audit_logs`
Repay	`loan_installments` (paid components, status) + `loans` (amount_paid, outstanding, maybe repaid) + `loan_transactions` (repayment + allocation) + `audit_logs`
Overdue job	`loan_installments` (overdue, penalty_due) + `loans` (penalty_fees, totals) + `loan_transactions` (penalty) + `audit_logs`

Framework tables (not detailed): `migrations`, `cache`, `cache_locks` (idempotency locks when Redis is absent), `jobs`, `job_batches`, `failed_jobs`, `sessions`, `password_reset_tokens`.

9 Data dictionary

Generated from the migrated MySQL 8.4 database (`information_schema`), so types, nullability, indexes and foreign-key actions are exactly what's deployed. PK primary key · FK foreign key · UQ single-column unique.

institutions

Tenants: the lenders (banks, MFBs, credit unions) running on the platform. Not tenant-scoped itself.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
name	varchar(255)			
slug	varchar(64)		UQ	Tenant identifier used in the X-Tenant header.
email	varchar(255)			
phone	varchar(32)	yes		
country_code	char(2)			ISO 3166-1 alpha-2.
currency	char(3)			ISO 4217 currency; copied onto every product.
platform_fee_bps	int unsigned	yes		10MG fee for this lender in basis points; NULL = platform default (PLATFORM_FEE_BPS).
platform_fee_bearer	varchar(16)	yes		lender or borrower; NULL = platform default (PLATFORM_FEE_BEARER).
status	varchar(16)			pending → active ↔ suspended.
approved_at	timestamp	yes		When a platform operator activated it.
suspended_at	timestamp	yes		When it was suspended (tokens are revoked).
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.
deleted_at	timestamp	yes		Soft-delete marker.

INDEXES

- **UNIQUE** (slug)
- **INDEX** (status)

institution_brandings

One-to-one white-label profile per institution, served to the public storefront.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK UQ	Owning tenant (unique: one branding per institution).
display_name	varchar(255)			
logo_url	varchar(2048)	yes		HTTPS logo URL.
primary_color	char(7)			Hex colour, e.g. #0B5FFF.
secondary_color	char(7)			Hex colour.
support_email	varchar(255)	yes		
support_phone	varchar(32)	yes		
website_url	varchar(2048)	yes		
custom_domain	varchar(255)	yes	UQ	White-label domain; globally unique.
created_at	timestamp	yes		Creation time.

Column	Type	Null	Key	Notes
updated_at	timestamp	yes		Last update time.

INDEXES

- **UNIQUE** (custom_domain)
- **UNIQUE** (institution_id)

FOREIGN KEYS

- institution_id → institutions on delete cascade

institution_kyc_settings

One-to-one KYC policy per lender: whether verification is required and which checks (BVN, NIN, phone match) it runs.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK UQ	Owning lender (unique: one policy per institution).
kyc_required	tinyint(1)			Borrowers must pass every enabled check before applying.
bvn_check_enabled	tinyint(1)			Match the owner's BVN with NIBSS.
nin_check_enabled	tinyint(1)			Match the owner's NIN with NIMC.
phone_match_required	tinyint(1)			A phone mismatch fails the check instead of only being recorded.
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **UNIQUE** (institution_id)

FOREIGN KEYS

- institution_id → institutions on delete cascade

users

Every login: platform operators (institution_id NULL), lender staff and borrower owners. Email is unique per institution.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)	yes	FK	Tenant; NULL only for platform operators.
name	varchar(255)			
email	varchar(255)			
role	varchar(32)			platform_admin, institution_admin, loan_officer, finance_officer, customer.
is_active	tinyint(1)			Deactivation also revokes all tokens.
email_verified_at	timestamp	yes		
password	varchar(255)			Bcrypt hash.
last_login_at	timestamp	yes		Last successful login.
remember_token	varchar(100)	yes		
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **UNIQUE** `(institution_id, email)`
- **INDEX** `(institution_id, role)`

FOREIGN KEYS

- `institution_id` → `institutions` on delete restrict

customers

Borrowing healthcare businesses. The linked user is the owner or primary contact.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
user_id	char(26)		FK UQ	Owner / primary contact login (unique).
business_name	varchar(255)			
business_type	varchar(32)			pharmacy, patent_medicine_store, clinic, hospital, laboratory, diagnostic_centre.
registration_number	varchar(32)			CAC number (RC/BN/IT...), unique per institution.
licence_number	varchar(64)			Operating licence from the regulator (PCN, MDCN, MLSCN, State MoH).
licence_expires_at	date			An expired licence forces a decline recommendation.
year_established	smallint unsigned			Drives years-in-operation scoring.
phone	varchar(32)			Unique per institution.
address	varchar(255)	yes		
city	varchar(100)			
state	varchar(100)			
staff_count	smallint unsigned			
monthly_revenue	bigint unsigned			Minor units (kobo).
monthly_expenses	bigint unsigned			Minor units; revenue − expenses = net cash flow.
kyc_status	varchar(16)			Outcome of the latest attempt: pending, verified, failed.
kyc_checks	json	yes		Checks passed so far, e.g. ["bvn","nin"]; compared with the lender's current settings.
kyc_verified_at	timestamp	yes		When both BVN and NIN passed.
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **INDEX** `(institution_id, business_type)`
- **UNIQUE** `(institution_id, phone)`
- **UNIQUE** `(institution_id, registration_number)`
- **UNIQUE** `(user_id)`

FOREIGN KEYS

- `institution_id` → `institutions` on delete cascade
- `user_id` → `users` on delete cascade

customer_directors

The people behind a borrowing business. Holds the owner's BVN and NIN (encrypted) and the NIBSS / NIMC verification outcome.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
customer_id	char(26)		FK	The business this person owns or directs.
role	varchar(16)			owner or director; unique per business.
first_name	varchar(100)			As submitted; matched against NIBSS and NIMC.
middle_name	varchar(100)	yes		Optional; matched when supplied.
last_name	varchar(100)			Must match the BVN and NIN records.
date_of_birth	date			Must match the BVN and NIN records.
phone	varchar(32)			Recorded; a mismatch with the BVN phone is tolerated.
bvn	text	yes		Bank Verification Number, encrypted (AES-256, APP_KEY). Never serialised. NULL when the lender does not run the BVN check.
bvn_hash	char(64)	yes		HMAC-SHA256 fingerprint; unique per institution (one BVN, one business).
bvn_last4	char(4)	yes		Last four digits for masked display.
nin	text	yes		National Identification Number, encrypted. Never serialised. NULL when the lender does not run the NIN check.
nin_hash	char(64)	yes		HMAC-SHA256 fingerprint; unique per institution.
nin_last4	char(4)	yes		Last four digits for masked display.
bvn_reference	varchar(64)	yes		NIBSS lookup reference.
bvn_verified_at	timestamp	yes		Set when NIBSS matched name and date of birth.
nin_reference	varchar(64)	yes		NIMC lookup reference.
nin_verified_at	timestamp	yes		Set when NIMC matched name and date of birth.
verification_checks	json	yes		Per-field matches and failure reasons from the latest attempt.
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **UNIQUE** (customer_id, role)
- **UNIQUE** (institution_id, bvn_hash)
- **UNIQUE** (institution_id, nin_hash)

FOREIGN KEYS

- customer_id → customers on delete cascade
- institution_id → institutions on delete cascade

loan_products

Each lender's configurable products: limits, pricing, repayment and overdue rules, eligibility.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
name	varchar(255)			
code	varchar(32)			Product code, unique per institution.
description	text	yes		
currency	char(3)			

Column	Type	Null	Key	Notes
min_amount	bigint unsigned			Minor units.
max_amount	bigint unsigned			Minor units.
min_tenor	smallint unsigned			Number of installments.
max_tenor	smallint unsigned			Number of installments.
interest_rate_bps	int unsigned			Annual rate in basis points (2400 = 24% p.a.).
interest_method	varchar(32)			flat or reducing_balance.
repayment_frequency	varchar(16)			weekly, biweekly or monthly.
grace_period_days	smallint unsigned			Days after due date before an installment is overdue.
late_fee_type	varchar(16)	yes		fixed (minor units) or percentage (bps of installment outstanding).
late_fee_value	bigint unsigned			Late fee amount or rate.
min_credit_score	smallint unsigned	yes		Product-specific decline threshold (default 560).
eligible_business_types	json	yes		JSON list of allowed business types; NULL = all.
is_active	tinyint(1)			Inactive products cannot be applied for.
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.
deleted_at	timestamp	yes		Soft-delete marker.

INDEXES

- **UNIQUE** (institution_id, code)
- **INDEX** (institution_id, is_active)

FOREIGN KEYS

- institution_id → institutions on delete cascade

loan_product_fees

Fees attached to a product, either deducted upfront or capitalised into the repayable amount.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
loan_product_id	char(26)		FK	References loan_products.
name	varchar(255)			
calculation	varchar(16)			fixed (minor units) or percentage (bps of principal).
value	bigint unsigned			Fee amount or rate.
charge_type	varchar(16)			upfront (deducted at payout) or capitalized (spread over installments).
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **INDEX** (institution_id)
- **INDEX** (loan_product_id)

FOREIGN KEYS

- institution_id → institutions on delete cascade
- loan_product_id → loan_products on delete cascade

loans

Applications and loans, with a snapshot of the product terms and the running balances.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
customer_id	char(26)		FK	References customers.
loan_product_id	char(26)		FK	References loan_products.
reference	varchar(32)		UQ	Human-readable reference, e.g. LN-20261105-7KQ2M9AB.
status	varchar(16)			pending, approved, rejected, cancelled, disbursed, repaid.
currency	char(3)			
principal_amount	bigint unsigned			Minor units.
tenor	smallint unsigned			Number of installments.
interest_rate_bps	int unsigned			Snapshot of product rate.
interest_method	varchar(32)			Snapshot.
repayment_frequency	varchar(16)			Snapshot.
fees_snapshot	json			JSON copy of the product's fees at application time.
grace_period_days	smallint unsigned			Snapshot of overdue rules.
late_fee_type	varchar(16)	yes		Snapshot.
late_fee_value	bigint unsigned			Snapshot.
total_interest	bigint unsigned			Minor units.
upfront_fees	bigint unsigned			Deducted from the payout.
capitalized_fees	bigint unsigned			Added to the repayable amount.
platform_fee_bps	int unsigned			10MG fee rate locked in at application.
platform_fee_bearer	varchar(16)	yes		Who pays 10MG's fee on this loan, locked in at application.
platform_fee_amount	bigint unsigned			10MG fee in minor units (recorded in platform_fees at disbursement).
penalty_fees	bigint unsigned			Late fees charged so far.
net_disbursement_amount	bigint unsigned			Principal − upfront fees (cash paid out).
total_repayable	bigint unsigned			Principal + interest + capitalised fees + penalties.
amount_paid	bigint unsigned			Running total, changed only under row lock.
outstanding_balance	bigint unsigned			total_repayable − amount_paid.
purpose	varchar(32)			inventory_restock, medical_equipment, working_capital, emergency_financing, facility_expansion, other.
purpose_note	varchar(500)	yes		
reviewed_by	char(26)	yes	FK	Approving/rejecting user.
reviewed_at	timestamp	yes		
decision_reason	varchar(1000)	yes		Approval note, override reason or rejection reason.
disbursed_by	char(26)	yes	FK	Releasing user (must differ from reviewed_by).
disbursed_at	timestamp	yes		
disbursement_reference	varchar(255)	yes		Payment-rail reference.
first_due_date	date	yes		
maturity_date	date	yes		
closed_at	timestamp	yes		

Column	Type	Null	Key	Notes
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **INDEX** (customer_id, status)
- **INDEX** (disbursed_by)
- **INDEX** (institution_id, status, created_at)
- **INDEX** (loan_product_id)
- **UNIQUE** (reference)
- **INDEX** (reviewed_by)

FOREIGN KEYS

- customer_id → customers on delete restrict
- disbursed_by → users on delete set null
- institution_id → institutions on delete restrict
- loan_product_id → loan_products on delete restrict
- reviewed_by → users on delete set null

credit_assessments

The scorer's explainable decision for a loan application (one per loan).

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
loan_id	char(26)		FK UQ	Assessed loan (unique).
provider	varchar(64)			Scoring provider id (mock-healthcare-scorer).
score	smallint unsigned			300–850.
risk_grade	char(1)			A–E.
recommendation	varchar(16)			approve, review or decline.
debt_service_coverage_bps	int unsigned			Net cash flow ÷ monthly repayment (15000 = 1.50x).
net_monthly_cash_flow	bigint			Revenue − expenses at assessment time.
max_recommended_amount	bigint unsigned			Largest principal keeping coverage at 1.50x.
factors	json			JSON list of {factor, impact, detail}: the explanation.
assessed_at	timestamp			
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **INDEX** (institution_id)
- **UNIQUE** (loan_id)

FOREIGN KEYS

- institution_id → institutions on delete cascade
- loan_id → loans on delete cascade

loan_installments

The repayment schedule generated at disbursement, with per-component due/paid tracking.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
loan_id	char(26)		FK	References loans.
sequence	smallint unsigned			1-based installment number, unique per loan.

Column	Type	Null	Key	Notes
due_date	date			Derived from disbursement date and frequency.
principal_due	bigint unsigned			
interest_due	bigint unsigned			
fee_due	bigint unsigned			
penalty_due	bigint unsigned			
total_due	bigint unsigned			principal + interest + fee + penalty due.
principal_paid	bigint unsigned			
interest_paid	bigint unsigned			
fee_paid	bigint unsigned			
penalty_paid	bigint unsigned			
amount_paid	bigint unsigned			
status	varchar(16)			pending, partially_paid, paid, overdue.
paid_at	timestamp	yes		
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **INDEX** (institution_id)
- **UNIQUE** (loan_id, sequence)
- **INDEX** (status, due_date)

FOREIGN KEYS

- institution_id → institutions on delete cascade
- loan_id → loans on delete cascade

loan_transactions

Append-only money ledger: disbursements, fees, repayments and penalties.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID).
institution_id	char(26)		FK	References institutions.
loan_id	char(26)		FK	References loans.
type	varchar(16)			disbursement, fee, repayment, penalty.
amount	bigint unsigned			Minor units.
reference	varchar(40)		UQ	Internal unique reference (TXN-...).
external_reference	varchar(255)	yes		Provider reference; unique per institution, so it can't be applied twice.
channel	varchar(32)	yes		
balance_before	bigint unsigned			Outstanding balance before the entry.
balance_after	bigint unsigned			Outstanding balance after the entry.
allocation	json	yes		JSON waterfall split per installment (penalty, fee, interest, principal).
performed_by	char(26)	yes	FK	Acting user (NULL for system jobs).
meta	json	yes		
created_at	timestamp			Creation time.

INDEXES

- **UNIQUE** (`institution_id, external_reference`)
- **INDEX** (`loan_id, created_at`)
- **INDEX** (`performed_by`)
- **UNIQUE** (`reference`)

FOREIGN KEYS

- `institution_id` → `institutions` on delete restrict
- `loan_id` → `loans` on delete restrict
- `performed_by` → `users` on delete set null

audit_logs

Append-only audit trail of every privileged action, with actor, request id and value diffs.

Column	Type	Null	Key	Notes
id	bigint unsigned		PK	Primary key (auto-increment).
institution_id	char(26)	yes	FK	Tenant (NULL for platform-level events).
user_id	char(26)	yes	FK	References users.
action	varchar(64)			Dotted event name, e.g. loan.approved.
auditable_type	varchar(255)	yes		Morph alias of the subject (loan, customer, ...).
auditable_id	varchar(26)	yes		
old_values	json	yes		JSON of changed attributes before.
new_values	json	yes		JSON of changed attributes after.
ip_address	varchar(45)	yes		
user_agent	varchar(512)	yes		
request_id	varchar(64)	yes		X-Request-Id of the originating request.
created_at	timestamp			Creation time.

INDEXES

- **INDEX** (`action`)
- **INDEX** (`auditable_type, auditable_id`)
- **INDEX** (`institution_id, created_at`)
- **INDEX** (`user_id`)

FOREIGN KEYS

- `institution_id` → `institutions` on delete set null
- `user_id` → `users` on delete set null

platform_fees

Append-only 10MG revenue ledger: the platform fee on each disbursed loan, owed by the lender to 10MG.

Column	Type	Null	Key	Notes
id	char(26)		PK	Primary key (ULID unless noted).
institution_id	char(26)		FK	References institutions.
loan_id	char(26)		FK UQ	The disbursed loan (unique: a loan is charged once).
bearer	varchar(16)			lender (commission) or borrower (upfront fee on the loan).
rate_bps	int unsigned			Rate locked in when the borrower applied (basis points of the principal).
base_amount	bigint unsigned			Principal the fee was calculated on (minor units).
amount	bigint unsigned			Fee in minor units.
currency	char(3)			ISO 4217, copied from the loan.
accrued_at	timestamp			Disbursement time; filters revenue reports.
created_at	timestamp	yes		Creation time.

INDEXES

- **INDEX** `(accrued_at)`
- **INDEX** `(institution_id, accrued_at)`
- **UNIQUE** `(loan_id)`

FOREIGN KEYS

- `institution_id` → `institutions` on delete restrict
- `loan_id` → `loans` on delete restrict

idempotency_keys

Stored responses for retried mutating requests, keyed per user. Pruned after 24 hours.

Column	Type	Null	Key	Notes
id	bigint unsigned		PK	Primary key (auto-increment).
user_id	char(26)		FK	References users.
idempotency_key	varchar(255)			Client-supplied key; unique per user.
request_method	varchar(10)			
request_path	varchar(2048)			
request_hash	char(64)			SHA-256 of method + path + canonical body.
status	varchar(16)			processing or completed.
response_status	smallint unsigned	yes		
response_body	longtext	yes		Stored response replayed on retry.
expires_at	timestamp			24 h after creation; pruned daily.
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **INDEX** `(expires_at)`
- **UNIQUE** `(user_id, idempotency_key)`

FOREIGN KEYS

- `user_id` → `users` on delete cascade

personal_access_tokens

Sanctum bearer tokens (hashed), polymorphic to users, with expiry.

Column	Type	Null	Key	Notes
id	bigint unsigned		PK	Primary key (auto-increment).
tokenable_type	varchar(255)			
tokenable_id	char(26)			Owning user (ULID morph).
name	text			
token	varchar(64)		UQ	SHA-256 hash of the bearer token.
abilities	text	yes		
last_used_at	timestamp	yes		
expires_at	timestamp	yes		Tokens expire after 30 days.
created_at	timestamp	yes		Creation time.
updated_at	timestamp	yes		Last update time.

INDEXES

- **INDEX** `(expires_at)`
- **UNIQUE** `(token)`
- **INDEX** `(tokenable_type, tokenable_id)`

10 Verification

Check	Result
Migrations on MySQL 8.4 (`10mg`)	All 10 migrations ran; demo data seeded (3 lenders, staff, a pharmacy and a clinic, 3 healthcare products, one loan through the full lifecycle)
Automated suite: SQLite in-memory	**260 tests · 1,151 assertions · all passing** (~5 s)
Automated suite: MySQL 8.4	**260 / 260 passing** (separate `10mg_testing` database)
Live HTTP flow checks	**56 / 56 passing**: onboarding, auth, branding, products, staff, borrower onboarding, eligibility, decline override, reject/cancel, four-eyes, idempotent retry, overpayment, payoff, overdue penalties, isolation, suspension, audit
Postman collection (Runner, 85 requests in 11 folders)	**85 / 85 passing** for a seeded lender with a lender-borne fee (lagos-mfb) and one with a borrower-borne fee (abuja-credit); every request asserts its status and key fields
Adversarial probe (`scripts/probe.sh`, 12-worker server)	**151 / 151 passing**: cross-tenant IDs, suspended lenders, disabled staff, segregation of duties, idempotency, duplicate references, overpayment, closed/cancelled loans, malformed input, deactivated products, open-loan limit, BVN/NIN KYC, lender KYC settings, and real parallel races
Data invariants (direct SQL)	For every disbursed/repaid loan: balance = schedule outstanding = last ledger balance = total repayable − paid

Test suite	Focus
`Unit/…/RepaymentScheduleCalculatorTest`	Flat vs reducing, EMI, exact rounding sums, fees, due dates
`Unit/…/MockHealthcareCreditScorerTest`	Licence, coverage bands, trading history, weekly normalisation, affordability
`Unit/Enums/*`	State machine; complete role → permission matrix
`Feature/TenantIsolationTest`	Cross-tenant 404s, header spoofing, fail-closed scope, per-tenant uniqueness
`Feature/RoleBasedAccessTest`	Maker/checker, staff vs borrower, self-demotion, deactivation
`Feature/Loan*Test`	Lifecycle, eligibility, repayments, disbursement failure, four-eyes
`Feature/IdempotencyTest` · `AuditTrailTest` · `MarkOverdueInstallmentsTest`	Replay, mismatch, concurrency, 5xx retry; audit immutability; penalties
`Unit/ArchitectureTest`	Layering rules for every controller and service; each repository contract implemented
`Feature/Services/StaffServiceTest`	Service rules tested against a mocked repository (no database)
`Feature/PlatformFeeTest`	Lender- vs borrower-borne pricing, quote disclosure, terms locked at application, waived fee, operator-only and audited, revenue per lender and across lenders, borrowers never see the commission, append-only
`Feature/KycSettingsTest`	Defaults, BVN-only never calls NIMC, re-enabling a check, KYC off, phone matching, invalid combination, per-lender isolation, admin-only, audit, storefront
`Feature/KycVerificationTest` · `IdentityRegistryClientTest`	KYC gate, masking and encryption, NIBSS/NIMC mismatch and retry, provider outage, one BVN per business, exact live request shapes, token caching

Test suite	Focus
`Feature/BusinessProfileTest · CustomerLoanHistoryTest`	Borrower self-service, licence renewal, immutable identity, staff loan history

11 Trade-offs & roadmap

Decision	Rationale / next step
Shared-schema tenancy	Cheapest to run for many tenants. All access goes through one tenancy layer, so a very large lender could later move to a dedicated database without business-code changes. Postgres row-level security could add a second, database-enforced layer.
Synchronous disbursement in the transaction	Fine for a mock rail. Production should use a *disbursing* state, a transactional outbox and a provider webhook. The deterministic gateway reference already makes rail-side retries safe.
Mock scorer, self-declared financials	The contract matches the real AI model. Next: ingest bank statements, POS settlements and wholesaler purchase history, verify licences against regulator registers, and re-score on each application.
Scheduled interest (no daily accrual)	Simple and exact for installment products. Next: daily accrual, early-settlement rebates and restructuring as product settings.
KYC for the owner only	Covers sole proprietors, most pharmacies. Each lender already chooses which checks run. Next: every director of a limited company, CAC director lookup, and re-verification when an owner changes.
One owner login per business	Enough for the assessment. Next: multiple staff per borrowing business (e.g. a pharmacy's accountant) via a membership table.
Platform fee without settlement	Fees accrue per loan and are reportable per lender. Next: monthly invoices to lenders and marking fees settled, with an optional platform-wide cap on lender product fees.
Scorer ignores outside debt	Existing exposure counts only loans with this lender. Next: a credit-bureau lookup (CRC, FirstCentral) keyed on the verified BVN, feeding outside debts and recent defaults into the score.
Platform roadmap	Event webhooks for lenders, per-tenant rate limits and notification templates, Terraform for the AWS layout in Figure 1.